

제18회 임베디드SW경진대회 개발계획서

[자율주행 모형자동차]

□ 개발 개요

○ 요약 설명

팀 명	CTRLCV
목 표	모형 자동차에 임베디드SW를 탑재하여 2차원 카메라를 통해 주변 지형 및 사물을 인식하고, 스스로 정해진 차로를 주행하며 주어진 미션을 수행하는 작품을 만드는 것
개발 내용 (요약)	인식-처리-제어의 3단계 모듈화, 코어 분할 기법으로 인식 단계 리소스 전담, 유한 상태 기계를 이용한 미션 모듈화, IMU 구현을 통한 차량의 현재 위치 파악, 강화 학습을 이용한 주차 알고리즘 개발
기술동영상	https://youtu.be/9iw8ru-MgAQ

○ 개발 목적

1. 지능형운전자보조시스템(ADAS) SoC(System on Chip)를 탑재한 모형 자동차를 개발해 봄으로써 차량용 프로세서에 대해 익숙해지고 실제 현장에 적용할 수 있는 능력을 기른다.
2. 2차원 카메라를 이용한 OpenCV 라이브러리 활용 능력과 영상 처리 능력을 기른다.
3. 리눅스 기반 임베디드 시스템에서 차량 제어 알고리즘 개발을 통한 자율 주행차의 개발과 임베디드 시스템에 대한 전반적 지식을 향상한다.
4. 다수가 함께 협업해야 하는 프로젝트에서 분업을 위해 철저히 모듈화하는 설계 방법을 익힌다. 이를 통해 팀원 간의 분업 및 협업이 활발히 일어날 수 있도록 하며, 충돌 가능성을 줄여 안정적인 소프트웨어를 구현한다.
5. 주변 상황을 수학적으로 모델링하고 모형 자동차에 적용함으로써 좀 더 안정적인 주행을 구현하고 수학적 사고력을 기른다.
6. 주행 중 나타날 수 있는 각종 장애물과 다양한 도로 형태에 유연하게 대처하는 로봇을 만들어봄으로써 실제 현장에서 어떠한 어려움이 있는지 직접 체험해 본다.

○ 개발 목표

1. 로봇의 3요소인 인식(Recognize), 처리(Process), 제어(Control) 과정을 모듈화한다.
2. 주어진 프로세서에 있는 두 코어 중 한 코어에만 인식 단계 Process를 할당하여 처리, 제어 단계로 인한 병목 현상을 줄일 수 있도록 한다.
3. 2차원 영상 처리는 적외선(Infrared, 이하 IR) 센서나 거리 센서(Position Sensitive Detector, 이하 PSD)에 비해 연산량이 매우 많으므로 적절한 필터를 사용하여 외부 환경에 빠르게 반응하고 매끄럽게 주행하는 로봇을 만든다.
4. 차선을 인식할 때 Perspective Transform, Binarization, Hough Transform, DBSCAN 등의 알고리즘을 이용하여 주변 상황의 영향을 최소화 시키고 정확히 차선을 인식할 수 있도록 한다.
5. 미션을 인식과 수행 상태로 나누어 이를 유한 상태 기계로 구현한다.
6. Internal Measurement Unit (IMU)를 영상 처리를 이용해 구현하고 차선이 잘 보이지 않는 경우나 주차 미션을 수행하는 경우에 활용할 수 있도록 한다.

□ 개발 방향 및 전략

○ 기술적 요구 사항

1. 자율주행 자동차 로봇의 개괄

모형 자동차가 스스로 도로 상황을 판단하고 주행하기 위한 기술적 요구 사항을 각 단계별로 나열했다. 먼저 인식(Recognize)-처리(Process)-제어(Control)의 3단계로 모듈화하여 소프트웨어를 개발하는 기술이 필요하다. 모듈화를 통해 팀원 각자가 각 단계를 개별적으로 구현할 수 있도록 한다. 각 단계의 기술적 요구 사항에 대해 자세한 내용은 아래의 “개발 방법”에서 다루도록 한다.

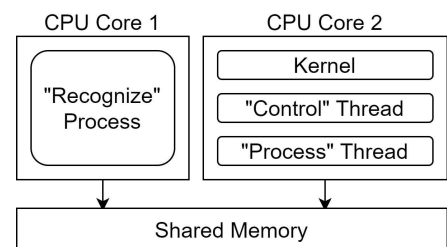


<Fig 1> 로봇의 3요소(인식-처리-제어); (화살표는 데이터의 전달을 나타냄)

- (1) 인식 단계에서는 2차원 카메라의 영상, PSD 센서 전압값 등으로부터 데이터를 수집한다. 그리고 이를 처리에 용이하도록 하기 위해 영상 처리 라이브러리인 OpenCV 등을 이용해서 가공한 후 처리 단계로 전달한다. 자세한 내용은 아래의 “영상 처리 및 센서 기술 활용 방안 및 공부 내용”에서 다룬다.
- (2) 처리 단계에서는 인식 단계에서 생성한 데이터를 분석해서 로봇을 어떻게 제어할 지, 미션을 어떤 알고리즘으로 수행할 지 등을 유한 상태 기계로 구현한다.
- (3) 제어 단계에서는 처리 단계에서 판단한 내용을 바탕으로 실제 동작을 수행한다. 특히 서보모터를 사용하기 위해 PWM을 통한 PID 제어를 수행한다.

2. 다중 코어 분할 사용 기법

인식 단계에서 영상 데이터를 가공하는 과정은 많은 컴퓨팅 자원이 필요하므로, 처리, 제어 단계가 느리게 동작하는 문제가 생긴다. 이를 해결하고자 리눅스 커널의 기능을 이용하여 CPU의 코어 하나를 완전히 독립시키고, 인식 단계 프로세스를 그 코어에 할당한다. 그러면 다른 작업과 인식 단계의 작업이 분리되어서, 다른 단계의 속도에 영향을 미치지 않게 된다. 그리고 이렇게 하려면 프로세스 간 통신이 필요한데, 이는 리눅스에서 제공하는 Shared Memory 기술을 사용한다. 인식 프로세스는 정보를 생성하여 Shared Memory 내의 Struct를 업데이트하고, 처리 프로세스는 이를 읽어 처리 과정을 수행한다.



○ 개발 방법

구분	정보
Fig 1-(a)	7조 IR 센서 정보, 6개 PSD의 ADC 정보, 2차원 카메라 영상, 엔코더 카운터
Fig 1-(b)	튜닝된 7조 IR 센서 정보, 6개 PSD의 거리 정보, 차선 정보, 도로 경사, 출발 flag, 우선정지 장애물 flag, 정지선 flag, 신호등 정보, 주행 거리 및 속도
Fig 1-(c)	목표 속도, 목표 조향각, 전조등/후미등/정지등 ON/OFF신호, 카메라 틸트 조향각
Fig 1-(d)	차량 속도 제어를 위한 DC 모터의 움직임, 차량 조향을 위한 서보모터의 움직임, 전조등/ 후미등/정지등 제어, 카메라 틸트 서보모터의 움직임

▲ Fig 1의 각 단계에서 생산하는 정보 또는 받는 정보

1. 인식 단계 개발

- (1) 모형 자동차와 장애물과의 거리 정보 출력: ADC로부터 PSD 센서 전압값을 불러오고 Curve Fitting을 통해 구한 전압-거리 변환 함수를 이용하여 거리 정보를 출력한다. 그리고

Impulse Noise를 제거하기 위해 Median Filter를 적용한다.

- (2) 주행 거리 및 속도 출력: 모터에 부착된 엔코더는 축이 일정한 각도만큼 회전하면 펄스를 발생시킨다. 그리고 시스템은 엔코더에서 발생한 펄스를 센다. 발생한 펄스 수(엔코더 count)와 바퀴의 반지름(r), 시스템 내부 타이머를 이용하면 자동차의 주행 거리 및 속력을 구할 수 있다. 이때 모형 자동차의 엔코더 분해능은 195 step/1회전이다. 자동차의 주행 거리 s 는 다음 식으로 구할 수 있다. $s = r \times (2\pi/195) \times (encoder\ count)$
- (3) IR 센서 정보 출력: 로봇의 앞쪽 아랫단에 있는 7조 IR 센서로 센서 아래가 밝은색인지 어두운색인지 감지한다. 바닥과 차선을 구분하는 임계값은 제공된 오토 튜닝 기능을 이용하면 되므로 신경 쓰지 않아도 된다.
- (4) 차선 정보: 카메라 영상 데이터로부터 차선 정보를 얻는다. 차선 정보는 차선의 개수와 각 차선의 위치, 곡률, 실선인지 점선인지 등을 포함한다. 차선 검출은 다음 과정을 따른다. Perspective Transform $\Rightarrow$ Grayscale $\Rightarrow$ Binarization with Adaptive Thresholding $\Rightarrow$ Hough Transform $\Rightarrow$ Reverse Perspective Transform. Hough Transform 대신 RANSAC등의 Regression을 적용할 수도 있다. 자세한 내용은 "영상처리 및 센서 기술 활용 방안 및 공부 내용"에서 다룬다.
- (5) 도로 경사: 판단한 차선의 기울기 정보를 바탕으로 소실점 위치를 계산하고, 이전 소실점 위치와 현재 소실점 위치의 differential을 통해 도로 경사의 변화율을 구한다.
- (6) 출발 신호 정보, 우선정지 장애물 정보 출력: 손동작 혹은 자체 제작 표지판 등 정해진 출발 신호와 우선정지 장애물 정보를 영상 처리로 확인하고 해당하는 Flag를 설정한다.

2. 제어 단계 개발

각 제어는 처리 단계에서 반환된 값을 통해 수행된다.

- (1) 목표 속력으로 제어: 모형 자동차의 DC 서보모터 속력을 목표 속력으로 제어한다.
- (2) 목표 조향각으로 제어: 서보 모터를 이용해 로봇의 앞바퀴의 조향각을 목표 조향각으로 제어한다.
- (3) 전조등/후미등/정지등 제어: 전등 전원을 GPIO를 통해 제어한다.
- (4) 카메라 틸트 조향각 제어: 카메라의 각도를 서보 모터로 유동적으로 제어한다.

한편, 제어 프로세스는 처리 프로세스와 같은 프로세서에서 병렬로 구동된다. 그러나 처리 프로세스는 단순히 정보를 생산하고, 제어 프로세스는 이를 소비하기 때문에 Race Condition은 발생하지 않는다.

3. 처리 단계 개발

처리 단계에서 수행하는 모든 과정은 유한 상태 기계 (Finite State Machine, FSM)로 구현한다. FSM은 모형 자동차가 가질 수 있는 각각의 독립적인 상태(State)로 이뤄져 있으며, 각 상태는 주행 및 미션 수행을 위한 구체적인 값(목표 속도, 목표 바퀴 조향 등)을 출력한다. 각 상태는 인식 단계에서 받은 데이터에 따라 다른 상태로 전이한다.



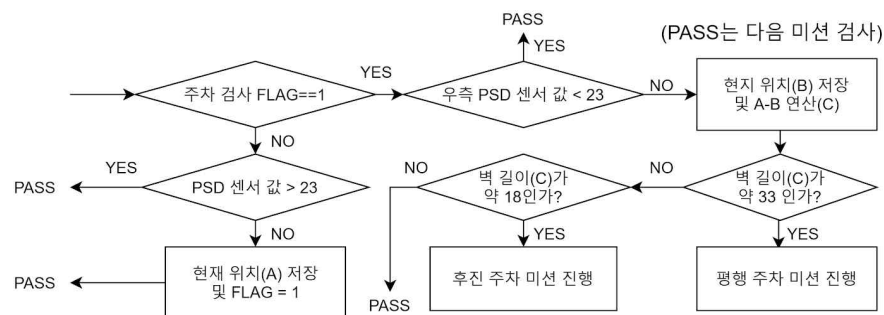
▲ 처리 단계의 FSM; (화살표는 작업의 순차적 흐름을 나타낸다.)

FSM은 상속 등을 통하여 재사용 가능한 형태로 작성한다. 특히, 각 State를 독립적으로 작성하여 각 State의 독립적 수행이 가능하도록 한다.

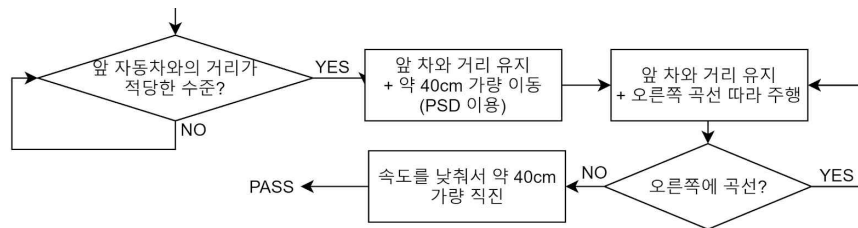
- (1) 대기 State (S0): FSM의 초기 상태로 무한 대기하면서 출발 신호가 들어올 때까지 정차한

다. 출발 신호가 감지되면 도로주행을 위한 준비를 마치고 S1(도로 주행)으로 전이한다. S0 상태를 벗어난 이후에는 다시 돌아오지 않는다.

- (2) 도로 주행 State (S1): 특별한 미션을 수행하지 않으며 평범하게 도로 위를 주행하는 상태이다. 직선과 곡선, 언덕, 고가도로 주행 등 회피할 장애물이 없거나 주차와 같은 특정 행동이 없는 구간에서 동작한다. 차선 정보를 바탕으로 자동차의 속력과 방향을 결정한다. 또한, 예기치 못한 상황으로 차선을 뺏게 되거나 가드레일에 부딪히는 경우를 감지하여 대응한다. 이러한 과정을 수행한 후에 반드시 S2(미션 검사)로 전이한다.
- (3) 미션 검사 State (S2): 특별한 동작을 요구하는 미션 구간을 판단한다. 인식 단계의 출력 데이터를 통해 어떤 미션 구간인지를 파악한 뒤 S3(미션 수행)로 전이한다. 미션 구간이 아니면 다시 S1로 전이한다. 검사할 미션은 아래와 같다.
 - (a) 우선 정지 장애물 구간: 우선 정지 장애물 flag가 1일 때 우선 정지 장애물 미션 구간으로 판단하고 S3로 전이한다.
 - (b) 평행 주차 및 후진 주차 구간

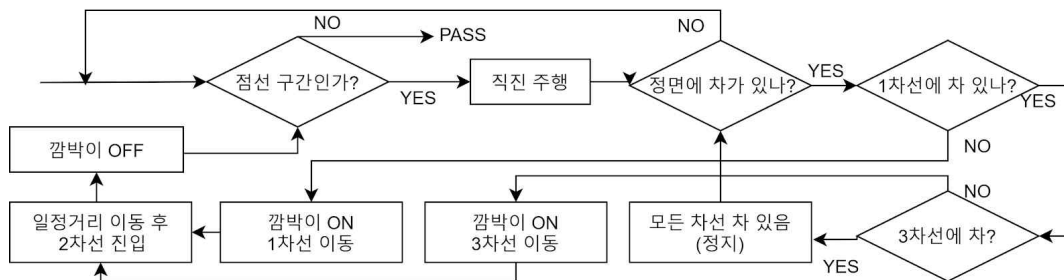


- (c) 회전 교차로 구간: IR 센서에서 정지선이 감지되면 일단 정지한다. 혹시 신호등 미션은 아닌지 정면 카메라를 통해 검사하고 아니면 차가 올 때까지 대기한다.
 - (d) 터널 구간: 좌우 PSD 인식 거리가 모두 15cm 이내이고 영상의 밝기가 일정 수준 이하로 낮아지면 터널 미션 구간으로 판단하고 S3로 전이한다.
 - (e) 차로 추월 구간: 점선을 발견하면 차로추월구간 미션 수행 S3으로 전이한다.
 - (f) 신호등 분기점 구간: 정지라인을 발견하면 우선 멈춘다. 카메라를 올려 신호등이 있는지 확인하고 있으면 S3으로 전이한다.
- (4) 미션 수행 State (S3)
- (a) 미션 검사 결과에 해당하는 미션을 수행한다. 미션 수행을 완료할 때까지는 S3상태에 머무른다. 수행이 완료되면 다시 주행을 위한 상태로 돌아간 다음, 미션을 완료하였음을 알리고 S4로 전이한다.
 - (b) 우선 정지 장애물 구간: 앞에 있는 장애물과의 거리를 계속 검사하면서 3~10cm이내일 때 정차한다. 우선 정지 장애물 flag가 0이 되고 일정 시간이 지나면 미션 성공으로 인식하고 S4 상태로 전이한다.
 - (c) 평행 주차 및 후진 주차 구간: 첫 번째로, 평행 주차 구간은 주차 공간 부근에서 방향을 오른쪽으로 두고 후진한다. 다시 방향을 가운데로 맞추고 약간 후진한 다음 방향을 왼쪽으로 바꾸고 후진한다. 주차 공간과 로봇이 나란하도록 우회전하여 전진하여 주차를 마무리한다. 두 번째로, 후진 주차 구간은 주차 공간 부근에서 방향을 왼쪽으로 하고 약간 진행 후, 방향을 오른쪽으로 바꾼 뒤 후진하여 주차한다. 주차 완료 후 부저를 울리고, 도로 한 가운데로 이동하여 주행 준비를 마치고 하고 S4로 전이한다.
 - (d) 회전 교차로 구간



(e) 터널 구간: 터널구간에 들어가면 우선 전조등을 켜다. 양쪽 PSD거리를 검사해서 계산한 위치와 2차원 카메라에서 받아들이는 차선 정보를 대조하여 주행한다.

(f) 차로 추월 구간:



(g) 신호등 분기점 구간: 미션 검사에서 카메라를 올려 신호등을 확인했다. 여기서는 영상처리로 신호를 알아낸 후, 카메라를 다시 내려 신호등이 지시한대로 좌/우회전을 하여 주행종료 구간에 정착한 다음, 주행을 완료한다.

(5) 미션 종료 State (S4): 다시 S1로 돌아가기 위한 준비 과정을 거치고 S1로 전이한다. 속도, 카메라의 각도, 그리고 각종 전등 등을 일반 주행상황에 적합한 조건으로 설정한다.

○ 예상되는 장애요인 및 해결방안

1. PSD센서 특성상 노이즈가 많다. ⇒ 임펄스 노이즈는 Median 필터로, 고주파 노이즈는 Low Pass Filter(LPF)로 보정하여 해결한다. Median Filter는 ADC부분에서 3개의 값을 받아 중앙값을 반환한다. 그리고 비트연산으로 하위 4비트의 작은 노이즈를 제거한다. 더 강력한 노이즈 제거는 Infinite Impulse Response(IIR)디지털 필터를 이용한 LPF를 사용한다.
2. 하드웨어 성능의 부족으로 비용이 큰 알고리즘(패턴매칭, 인공지능)이 제대로 작동하지 않을 수 있다. ⇒ 패턴매칭은 이진화와 resize를 통해 연산량을 줄이고, AI 역시 neural network보다는 고전적 알고리즘에서 gain tuning만을 강화학습으로 학습시키는 방법을 사용한다.
3. 미션과 관련된 함수코드를 모두 합쳤을 때 문제가 발생할 수 있다. 특히 비슷한 종류의 입력으로 미션 검사를 하는 미션들 간의 우선순위를 잘 정해야 한다. 우선정지 장애물은 도로 주행 구간에서 나타나며, 다른 미션보다 높은 우선순위를 주어 장애물 발견 시 즉시 멈출 수 있도록 한다.
4. 포맥스 바닥에서 바퀴가 미끄러진다. 엔코더 카운터도 별로 쓸모가 없어지고 미끄러지는 방향과 반대로 방향을 틀면 오히려 차선을 탈출할 수도 있다. ⇒ 카운터 스티어링기법을 적용한다. 이는 후륜 자동차에서 미끄러지는 방향으로 약간 방향을 틀어 원래의 주행 방향을 되찾는 기법이다.

○ 예상 결과 작품이 활용될 분야 및 방법 제시

작은 소포들을 배달하는 무인 자동차에 먼저 적용될 수 있다. 사람이 탑승한 자동차는 이번 프로젝트의 작품보다 안전이나 다양한 돌발 상황에 더욱 유의해야 하지만, 특정 지역을 벗어나지 않는 배달 자동차는 이번 작품과 유사한 점이 많다. 혹은 응급상황에서 119대원이 도착하기 전에 가까운 배달 자동차가 현장에 도착해 미리 응급키트를 보내는 용도로도 사용할 수 있다. 또한 관리자가 직접 현장에 방문하지 않아도 최적의 배달경로로 물건을 옮길 수 있으므로 노동력이 절약된다.

□ 영상처리 및 센서 기술 활용 방안 및 공부 내용

○ 영상처리 기술 활용 방안 및 공부 내용

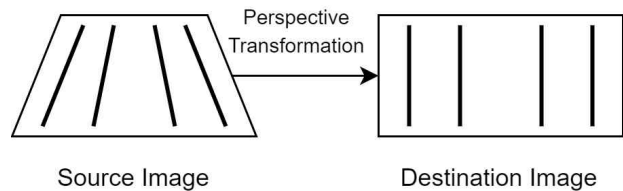
OpenCV는 영상을 통한 차선 검출 또는 물체 인식 알고리즘 구현을 위한 영상 처리 (Computer Vision) 라이브러리이다. 카메라로부터 받은 정보를 OpenCV의 도움을 받아 가공한 후 다음의 영상 처리 기술 또는 알고리즘을 적용해 볼 수 있다. 이를 활용해서 차선이나 물체를 인식할 수 있고 더 나아가 현재 위치를 알아낼 수 있다.

- Perspective Transform
- Edge Detection: Canny, Sobel, Adaptive Thresholding, Otsu's Thresholding
- Line Detection: Hough Transform, RANSAC, DBSCAN Clustering
- RGB to HSV Conversion, Contour Detection
- Optical Flow, Homogeneous Coordinate Transformation

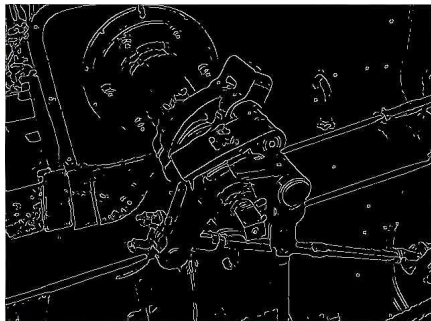
영상 처리는 차선 인식, 노면 각도 인식, 신호등 인식, 위치 인식으로 나눌 수 있다.

1. 차선 인식

차선을 인식하기 위하여 이미지를 Perspective Transform을 사용하여 항공 뷰로 본 것처럼 바꾼다. 사용되는 상수는 카메라의 성능 및 위치를 고려하여 미리 구한다. 이후 이미지에서 차선만을 추출하는 알고리즘을 사용한다. Edge Detection, Adaptive Thresholding, Otsu's



▲ Perspective Transformation 예시

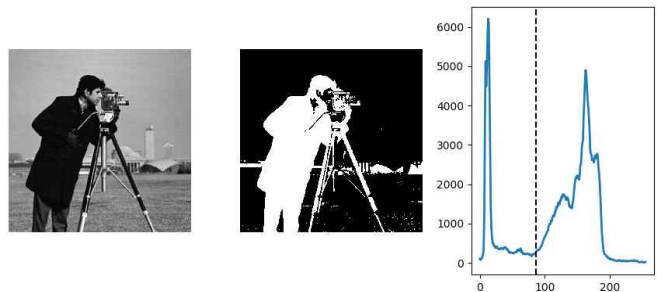


▲ Edge Detection을 수행한 사진

Thresholding 등 다양한 방법이 사용될 수 있다.

Edge Detection은 경계를 검출하는 방법인데, Canny, Sobel 등의 유명한 검출 알고리즘이 있다. 다만 노이즈에 취약하고, 차선이 아니더라도 경계선이면 전부 인식한다는 단점이 있다. Adaptive Thresholding은 이미지에 그림자 등 부분적인 변화가 있어 전경과 배경을 하나의 상수로 구분하기 어려울 때 사용하는 방법으로, 판단할 픽셀과 그 주변 픽셀을 비교하여 전경인지 배경인지를 결정한다. 그러나 배경 영역의 국소적 변화에 민감하여 배경에 나타나는 경계

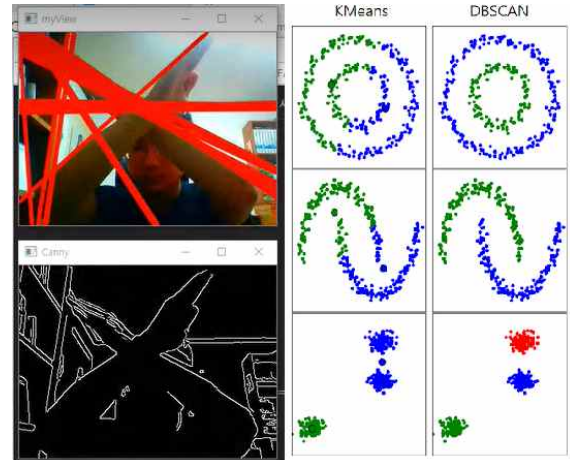
를 전경으로 보는 일이 많다. Otsu's Thresholding은 이미지에 색상 클래스가 두 개 있다고 가정한 후, threshold에 따른 클래스의 공분산을 최대화하는 threshold를 찾아+내는 알고리즘으로, 성능이 매우 우수하나 global thresholding 알고리즘이므로 배경에 그림자 등이 있으면 검출 능력이 떨어진다. 따라서 먼저 이미지를 다항식 근사한 것을 이미지에 써 빼서 전역적 변화를 제거한 후 Otsu's Thresholding을 사용하면 차선을 비교적 정확하게 구할 수 있다.



▲ Otsu's Thresholding (右: 적당한 Threshold를 잡는 모습)

다음으로 이렇게 구해진 차선 영역을 주행에 사용할 수 있도록 수식으로 바꾸는 과정이 필요하다. 대표적으로 Hough Transform 또는 RANSAC을 사용할 수 있다. Hough Transform은 이미지의 각 점을 지나는 모든 직선을 parameter space로 옮긴 후 중복도가 높은 점들만을 골라

image space의 직선으로 돌리는 알고리즘이다. 다만 차선은 이론상의 직선과 다르게 두께가 있어 한 차선에 여러 직선이 생성되는 문제가 생긴다. 따라서 DBSCAN Clustering 알고리즘을 사용해서 parameter space상의 가까운 점들을 클러스터링한 후, 각 클러스터의 중간값이 되는 직선을 골라서 차선으로 간주한다. 여러 Clustering 알고리즘 중 DBSCAN을 사용하는 이유는(Expectation Maximization이나 K-means 등과 다르게) 클러스터 개수를 미리 지정해줄 필요가 없고, 노이즈를 잘 구분하며, 복잡한 형태의 클러스터링을 할 수 있고, 계산 속도가 빠르기 때문이다.



▲ Hough Transform, ▲ Clustering Algorithm

2. 노면 각도 인식

이제 차선을 인식하였으므로 다음 식을 사용하여 노면 각도를 알아낼 수 있다.

$$\theta_{cam} = \tan^{-1} \left(\frac{y_{vanish}}{h/2} \tan \left(\frac{\theta_{view}}{2} \right) \right)$$

여기서 θ_{cam} 은 노면에 대한 카메라 광축의 각도, y_{vanish} 는 차선 두 개가 만나 이루는 소실점의 위치(화면 중심이 원점), h 는 화면 높이, θ_{view} 는 카메라의 종방향 화각이다.

3. 신호등 인식

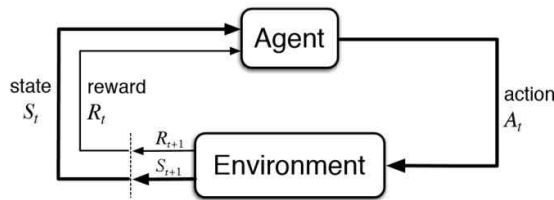
신호등을 인식하고자 영상을 HSV 색공간으로 바꾼다. 이는 HSV 색공간에서는 명도에 관계없이 색조만을 판단할 수 있어 명도 변화가 있을 때 색 검출에 유리하기 때문이다. HSV 색공간으로 바꾼 이미지에 대하여, 붉은색, 노란색, 초록색에 해당하는 색조, 채도, 명도의 범위를 지정하여 원하는 영역을 추출한다. 그리고 이에 대한 Contour를 구하여 도형의 모양을 인식한다. 여기에서 특정 Contour를 다각형에 근사했을 때 나타나는 변의 수로 화살표와 원을 구별한다. 변이 7개이면, 그 도형의 무게중심의 왼쪽에 점이 2개이고, 오른쪽에 점이 5개라면 그 도형은 오른쪽 화살표이고, 반대로 왼쪽에 점이 5개이고, 오른쪽에 2개라면 왼쪽 화살표임을 추측할 수 있다. 마지막으로 근사한 다각형의 변이 7개를 초과한다면 그 도형을 원이라고 추측한다. 이렇게 하면 신호등의 4가지 신호인 원과 화살표를 모두 감지할 수 있다.

4. 위치 인식

다음으로 위치 인식은 일종의 Internal Measurement Unit(IMU)를 구현하는 것이다. IMU란 외부의 위성을 사용하는 GPS등과 다르게 차량 내부의 장치만으로 위치를 파악하는 기술이다. 보통 Gyroscope 및 Accelerometer를 많이 사용한다. 다만 우리 차량에는 그런 장치가 없으므로 영상처리를 통해 구현한다. 원리는 간단한 패턴 매칭을 통한 Optical Flow를 이용하는 것으로, 한 프레임을 격자로 분할하여 각 격자가 다음 프레임에서 어디로 이동했는지를 구한 후, 그 이동의 변환 행렬을 구하면 된다. 다만 프레임의 위치는 2차원이고, 2차원에서는 행렬을 통한 평행이동을 구현할 수 없다. 따라서 실제로는 여분의 차원을 하나 더하는 Homogeneous Coordinate를 사용하여 구현한다. 이 변환 행렬은 차량의 위치에 대한 노면의 위치 변화를 뜻한다. 그러므로 이 행렬의 역행렬을 구하면 노면의 위치 변화에 대한 차량의 위치 변화를 구할 수 있다. 이제 이를 적분하면 차량의 현재 위치를 얻는다. 이는 차선이 잘 보이지 않을 수 있는 급격한 커브길이나 주차 시에 사용될 수 있다. (다만 아무런 최적화 없이 구현하면 한 프레임당 계산량이 10^{11} 클럭정도로, 대략 100초정도 시간이 걸린다. 따라서 예상되는 장애요인 및 해결방안에서 제시한 최적화 방법을 사용할 것이고, 그러면 이론상 계산량이 70,000클럭정도까지 줄어든다. 이는 1~10ms내로 처리할 수 있는 정도이다.)

○ 센서 기술 활용 방안 및 공부 내용

1. Reinforcement Learning (RL)을 이용한 미션 수행



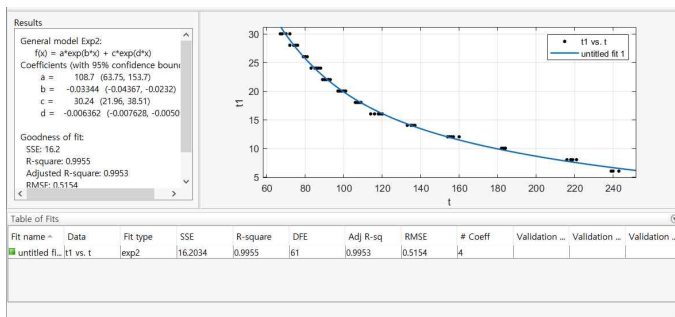
▲ Diagram of Reinforcement Learning

시중에 나오는 자동차는 자동 주차 시스템이 탑재되어 나오는 추세이다. 모형 자동차 또한 인공지능의 일종인 강화학습(Reinforcement Learning)을 이용해 자동 주차 알고리즘을 구현해 볼 수 있다. Agent는 모형 자동차, Observation은 PSD/IR센서 데이터, Environment는 현실 세계, 그리고 Reward는 올바른 주차 과정을 수치적으로 정량화 한 것

이라고 할 때, Reward를 최대화 하는 Action을 취하도록 하는 Policy π 를 찾는다. Policy란 현재 상황에서 어떤 Action을 취할지 결정하는 방법을 말하며, 자율주행차의 경우 예를 들어 전진, 후진, 조향, 정지 중 하나를 고를 수 있다. 다만 Policy를 찾는 과정은 수많은 반복이 필요하여 실제 자동차로 훈련한다면 시간이 상당히 오래 걸릴 것이다. 따라서 컴퓨터 시뮬레이션을 통해 실제와 유사한 Policy π' 을 대략적으로 찾고, 이 π' 를 토대로 실제 자동차의 Policy를 Fine Tuning한다.

2. Position Sensitive Detector (PSD): 거리 측정 센서

특정 미션을 수행하기 위해 모형 자동차의 앞, 뒤, 측면과 장애물 사이의 거리를 구할 필요가 있다. 이때 Position Sensitive Detector (PSD)의 데이터를 활용할 수 있다.



▲ cftool of MATLAB

PSD란 삼각 측량 방식을 사용한 센서로, 감지 대상의 색과 반사율에 영향을 거의 받지 않는다. 센서에서 감지 대상까지의 거리에 대응하는 전압이 출력되며 유효 구간(약 4~30cm)에서 지수함수의 양상을 따른다.

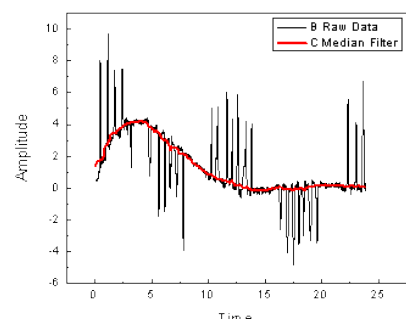
모형 자동차에는 총 6개의 PSD 센서가

존재하는데 각 PSD마다 특성이 다르기 때문에 이를 적절히 사용하기 위해 전처리를 해주어야 한다. 이를 위한 기술로 Curve Fitting이 있다.

Curve Fitting이란 실험적으로 구한 이산적인 데이터들을 충분히 잘 설명하는 이상적인 곡선 함수를 구하는 과정이다. Curve Fitting을 통해 PSD의 거리-전압 함수를 구할 수 있다. MATLAB의 cftool을 이용할 수 있으며 위에서 언급했듯이 PSD마다 특성이 다르기 때문에 거리-전압 함수를 각각 구한다. 그리고 이렇게 구한 함수를 모형 자동차 소프트웨어에 적용해서 더욱 정확한 거리를 측정할 수 있도록 한다.

이때 전압 정보는 아날로그 신호이기 때문에 이를 디지털로 변환하는 과정을 거쳐야 한다. 이는 전압을 디지털 신호로 변환하는 Analog Digital Converter (ADC)라는 회로를 통해 수행된다.

한편, PSD 센서의 오작동으로 인해 값이 팍 튀는 Impulse 노이즈가 생길 수 있으므로 Smoothing 기법인 Median Filter를 적용하여 이를 제거한다. 또한 측정 상에 눈에 띄는 노이즈가 발견될 경우 Low Pass Filter를 적용해서 이를 제거하도록 한다.



▲ Median Filter

□ 개발 일정

No	내용		2020년															
			6월				7월				8월				9월			
			1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1	프로세서 기능 및 스펙 분석																	
2	주변 장치(모터, PSD, 카메라)의 기능 및 스펙 분석																	
3	연습용 맵 제작																	
4	개발 환경 구축																	
5	인식 단계 개발																	
6	처리 단계 FSM 개발	기본적인 FSM 개발																
7		도로 주행 State 개발																
8		미션 검사 State 개발																
9		미션 수행 State 개발																
10	제어 단계 개발																	
11	모듈 통합 및 디버깅																	
기본적인 개발 완료																		
12	지속적인 최적화																	
13	강화 학습을 통한 주차 미션 수행																	
14	카메라를 사용한 IMU 구현(모형 자동차의 현재 위치 파악 알고리즘 구현)																	
추가적인 개발 완료																		
15	개발 완료 보고서 작성 및 제출																	

□ 팀 구성 및 역량

(소스코드는 하단 주석 참고 바람)

No	구분	성명	팀 내 담당 업무	업무 관련 역량 (개발 언어, 프로젝트 경험 등)
1	팀장	이성호	개발 환경 구축, 기술 문서 제작, 인식 단계 개발, 제어 단계 개발, 알고리즘 총괄	개발 언어: C, C#, Java, Python 프로젝트 경험: Kaggle Bengali Handwritten Recognize Competition(2019, 5인) ⁽¹⁾ , Moving trash can that recognizes and explores map(2019, 4인, 웹 어플리케이션 제작 및 임베디드 하드웨어 개발) ⁽²⁾
2	팀원	권준호	기본적인 FSM 개발, 도로 주행 State 개발, 카메라를 이용한 IMU 구현, 전체적인 시스템 성능 분석	개발 언어: C, C#, Java, Python, Javascript 프로젝트 경험: 서울시립대학교 국제도시과학대학원 재난 대피 Digital-Twin 국가 연구과제 참여(3인)(현재 진행 중), 서울시립대학교 인간중심인공지능연구실 인공지능솔루션팀 X-TWICE 국가 연구과제 참여(5인)(현재 진행 중), Moving trash can that recognizes and explores map(4인) ⁽²⁾ , 성균관대학교 자율주행자동차 융합설계경진대회 참여(3인), Machine learning method for position control of a vehicle-type robot with non-holonomic system(1인), 관찰자의 위치에 무관한 3차원 영상의 2차원 표현에 대한 탐구(1인) ⁽³⁾
3	팀원	김정현	신호등 분기점 구간 미션 수행 구현, 평행 주차 및 후진 주차 미션 수행 구현, 회전 교차로 구간 미션 수행 구현	개발 언어: C#, C++, Python 프로젝트 경험: Moving trash can that recognizes and explores map(2019, 4인, 맵 탐색 및 최단 경로 산출 알고리즘 등 동작 알고리즘 작성) ⁽²⁾ , 문정고등학교 학생 출석 관리 보조 서비스(백엔드, 프론트엔드) 개발 (2019, 1인) ⁽⁴⁾ , 2019 ACM-ICPC 참가 (3인), 2020 Google Hashcode 대한민국 32위 (3인, 세계 2200위)
4	팀원	박재선	주변 장치 보정 및 필터 적용, 터널 구간 미션 수행 구현, 우선 정지 장애물 구간 구현	개발 언어: C, Java 프로젝트 경험: 2019 SRC IRC SEOULTECH 지능로봇 - 지능형 씨름로봇 (2019, 4인), IR 센서로 경주로봇의 출발/도착 시간을 측정하는 계수기 하드웨어, 그리고 그와 유/무선 통신하는 소프트웨어(2019, 2인)
5	팀원	이준서	차로 추월 구간 미션 수행 구현, 오르막 및 내리막 구간 미션 수행	개발 언어: C, Verilog 프로젝트 경험: IR센서와 Step motor, ARM Processor를 이용한 Linetracer 제작 프로젝트(1인)

소스 코드 링크

(1) <https://www.kaggle.com/shythm/bengali-recognize/notebook> (3) <https://github.com/unknownpqr/auto-projection-matching>

(2) <https://github.com/unknownpqr/trashcan> (4) https://github.com/powerqee/MunjeongHS_Attendance/